Odyssey Catan Agent

Team Odyssey / Technical report / 17 October 2026

We built a Python agent that combines resource-aware evaluation, joint opening placement and bounded same-turn search. A small learned value model adds an outcome estimate to the heuristic. The frozen learned agent won **797 of 1,000 games against F (79.7%)** through the unmodified official match harness, with zero recorded faults or clock overruns for either player. This is a local Windows benchmark. Official O, private-set strength and group robustness remain unmeasured for this snapshot.

Objective and operating constraints

The task is two-player Catan to 10 victory points, without player-to-player trading. We select an index from the supplied legal actions. The agent sees its own development cards, the opponent resource hand and public board state. Opponent development-card identities remain hidden. The CPU allowance grows by decision count and limits how much planning can help [1].

Allowance after k decisions = min(300 + 12.5k, 1,500) ms. We therefore spend computation on opening choices and executable build sequences, while limiting the depth and width of ordinary search. Runtime inference uses one process and no external services.

Verified release

Measure	Frozen learned agent
Local wins against F	797 / 1,000 (79.7%)
95% interval by board pair	77.18% to 82.22%
Mean / worst local CPU per game	393.4 ms / 1,437.5 ms
Faults / games over allowance	0 / 0
First-seat / second-seat wins	394 / 500 and 403 / 500

Main contribution

The strongest evidence supports the combined heuristic and search system. A separate 2,000-game test of the heuristic agent yielded 79.6%. The learned release yielded 79.7% on different fresh boards. That 0.1 percentage-point difference does not establish an improvement from learning. Our contribution is a clock-bounded decision system with traceable evidence and an explicit account of what the experiments do and do not support.

The release source, model parameters and per-game records accompany the repository. Later experiments running outside this frozen snapshot are excluded from the report.

Decision method

Opening choices establish the resource economy

We score pairs of initial settlements, rather than judging the first location in isolation. Expected production comes from the dice probabilities of adjacent tiles. A concave resource utility rewards coverage and gives diminishing returns to repeated production. Port access, nearby expansion and the second settlement starting hand provide smaller adjustments. An approximate greedy model of F predicts which locations may disappear before our next placement. A weighted opponent-quality term discourages openings that give away an especially strong economy. This opponent model is a heuristic, not an exact simulation of F.

Afterstate evaluation

For each legal action, we copy a sampled world, execute the move and score the resulting state. The score includes actual own points, opponent public points, production, robber effects, bank and port exchange rates, usable resources, expansion distance, road potential and army progress. Production value decreases as either player approaches the finish. A completed win receives a dominant score.

V(s) = points + economy + hand readiness + expansion + awards + 0.4 f(s).
Here f(s) is the fitted tree-model logit. We add it only outside opening placement. The feature extractor includes only information available to the player. It uses the opponent development-card count, never the concealed card types.

Bounded search and tactical guarantees

Ordinary same-turn search retains up to six candidates at each level, explores up to five action levels and limits expansion to 160 successor nodes. A CPU deadline limits the follow-up search to at most 18 ms, scaled down using the remaining allowance. Root action scoring and opening placement still incur their own cost. Board-dependent quantities are cached using buildings, roads, robber position and the free-road phase.

At nine points, legal winning builds take priority. A focused bank-trade planner also checks whether a short exchange sequence can enable the final build. This covers a concrete failure mode: rejecting the first trade because it looks worse before a second trade completes a winning city. Below 40 ms of remaining allowance, a deterministic legal-action fallback avoids further sampling and search.

Development cards under uncertainty

A sampled deck must not make a purchase appear valuable simply because its next card happens to be a victory point. We score a development-card purchase by averaging over the public unseen card multiset. Combining the sampled deck with the sampled opponent hand recovers that observable multiset without revealing the real hidden hand. The search stops at the purchase rather than expanding a fortunate sampled result. Tactical knight timing also considers whether the robber blocks our production.

Learning and evaluation

A compact model with reproducible inference

We generated 1,000 offline games between the strong heuristic policy and F, alternating their seats. End-of-turn positions from both perspectives produced 124,344 labeled observations with 91 features. Labels indicate the eventual winner. Features describe permitted points, production, resources, ports, robber exposure, structures, award progress and build affordability. Each player has an isolated random state so search randomness does not change the simulated dice stream.

Whole games with game index divisible by five form the validation split. The remaining games fit a histogram gradient-boosted classifier with 64 trees, depth at most three, at most seven leaves, learning rate 0.1 and L2 regularization 15. The held-out-game AUC is 0.8848 and log loss is 0.4285. These measure outcome prediction, not playing strength. The exported model matches the fitted implementation within 1e-9 on 100 held-out vectors.

Matched candidate selection

400-game common screen	Wins	Win rate
Heuristic control	316	79.00%
Heuristic plus 0.4 logit	318	79.50%
Heuristic plus 0.9 logit	317	79.25%

All three completed screens had zero faults and zero clock overruns. The predeclared rule selected the highest-win clean candidate, the 0.4 blend. A two-win lead is weak evidence. We froze that source before the fresh 1,000-game evaluation and report its result independently.

Fresh-board protocol

The official match implementation and F remained unchanged. Each board is played twice with seats exchanged and the same dice stream. Fixed 40-game batches keep game indices and agent lifetimes independent of worker count. The final learned run uses board seeds 990000 through 990499 and a separate local dice key. Process clocks apply to both players. Manifests record Python 3.11.17, source hashes and harness hashes. Completion checks require every requested unique game.

The reported interval treats each two-game board pair as a cluster, accounting for the paired design. The earlier heuristic holdout won 1,592 of 2,000 games (79.6%) with no faults or overruns and 316.3 ms mean local CPU. Its boards differ from the learned holdout, so these two estimates are descriptive rather than a paired ablation. Local hardware timing does not establish timing on the organizer machine.

Evidence and lessons

Experiments that changed our decisions

Wider search did not earn its cost. On a common 400-game screen, a compact beam won 331 games at 336.8 ms mean CPU, versus 329 at 528.6 ms for the broad beam and 327 at 443.9 ms for the wide beam. The small win differences are uncertain, while the observed compute cost is clear. We retained bounded search rather than assuming more expansion would help.

A plausible card heuristic failed its comparison. We suspected excess development-card buying after securing Largest Army. The control won 334/400 games. Reducing the value of spare knights won 326/400, and the broader tactical-card reweighting won 323/400 on the same screen. Neither beat the control, so these variants stayed out of this release.

Prediction quality did not prove policy improvement. The value model achieves 0.8848 AUC, yet the common gameplay screen gains only two wins with the chosen blend. Outcome labels are noisy and sampled from the heuristic-versus-F state distribution. Better classification on those states need not rank alternative actions better. A larger, paired gameplay comparison is needed.

Reliability and limitations

Five behavioral checks cover legal immutable openings, a two-trade winning city, deck-order-independent card expectation, a distinct cache entry for the free-road phase and a legal low-clock fallback. The frozen 1,000-game run records no exceptions, illegal-action fallbacks, timeouts, crashes or clock overruns for either side. The runtime ZIP contains agent.py alone because the 64 tree parameters are embedded as ML_BIAS and ML_TREES. A separate JSON export supports inspection.

We have not measured this release against O or the group opponents. Its F-aware opening model may specialize to F. The learning data covers one matchup, and same-turn search does not model a full sequence of future dice rolls and opponent replies. Priorities are broader opponent coverage, better action-ranking labels and continued testing under the actual competition clock. This work does not establish a 90% win rate or a leaderboard rank.

Public material and development assistance

We used the official kit and F implementation [1], the pinned catanatron engine and graph code [2], and scikit-learn with NumPy and SciPy for offline learning [3]. Python and psutil support the evaluation tools. Language-model assistants helped write code and analyze experiments. Runtime play makes no language-model or network calls. Earlier repository experiments and their additional reading remain in the history directory.

References

[1] Official problem statement, evaluation kit and requirements.
[2] Catanatron, pinned commit d3f4ad05.
[3] Scikit-learn HistGradientBoostingClassifier.
Run manifests, raw game records and exact source copies: repository evidence/.